

HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 18

April 2, 2026

Spring Recess

Week 8 - Extended Example: Training Large Language Models



Week 9 - Extended Example: Distributed Linear Algebra



Week 10 - Extended Example: Molecular Dynamics



Week 11 - Extra Topic: Kokkos



Week 12 - Project Work Week



Week 13 - Review



EXAMPLE SETUP

- Pull the latest Lecture Examples

Code Server - COMPSCI 2050 (CPU)

This app will launch a [VS Code](#) server using [Code Server](#).

Number of hours

Number of CPUs

Launch

* The Code Server - COMPSCI 2050 (CPU) session data for this session can be accessed under the [data root directory](#).

The screenshot shows the GitHub interface for the 'lecture-examples' repository. The repository is owned by 'CS-2050' and is marked as 'Internal'. It has 1 issue, 0 pull requests, 0 actions, 0 projects, 0 wiki pages, and 0 watchers. The repository is currently on the 'main' branch. The file list shows several lecture folders and a README.md file. The most recent commit is by 'wcw398' 2 days ago, with 14 commits in total.

File/Folder	Commit Message	Commit Time
lecture-02	add lecture-02.	last month
lecture-04	lecture-04 examples.	last month
lecture-06	.	last month
lecture-07	.	last month
lecture-08	.	last month
lecture-11	add lecture-11/example-3 solutio...	3 weeks ago
lecture-12	.	3 weeks ago
lecture-15	.	2 days ago
README.md	.	3 weeks ago

ACEpotentials.jl: A Julia implementation of the atomic cluster expansion

Cite as: J. Chem. Phys. 159, 164101 (2023); doi: 10.1063/5.0158783

Submitted: 17 May 2023 • Accepted: 25 August 2023 •

Published Online: 23 October 2023



View Online



Export Citation



CrossMark

William C. Witt,¹ Cas van der Oord,² Elena Gelžinytė,² Teemu Järvinen,³ Andres Ross,³ James P. Darby,⁴ Cheuk Hin Ho,³ William J. Baldwin,² Matthias Sachs,⁵ James Kermode,⁴ Noam Bernstein,⁶ Gábor Csányi,^{2,a)} and Christoph Ortner^{3,a)}

AFFILIATIONS

¹Department of Materials Science and Metallurgy, University of Cambridge, Cambridge, United Kingdom

²Engineering Laboratory, University of Cambridge, Cambridge CB2 1PZ, United Kingdom

³Department of Mathematics, University of British Columbia, 1984 Mathematics Road, Vancouver, British Columbia V6T 1Z2, Canada

⁴Warwick Centre for Predictive Modelling, School of Engineering, University of Warwick, Coventry CV4 7AL, United Kingdom

⁵School of Mathematics, University of Birmingham, Birmingham B15 2TT, United Kingdom

⁶Center for Materials Physics and Technology, U.S. Naval Research Laboratory, Washington, District of Columbia 20375, USA

Note: This paper is part of the JCP Special Topic on Software for Atomistic Machine Learning.

a) Authors to whom correspondence should be addressed: gc121@cam.ac.uk and ortner@math.ubc.ca

Current members



Alan Edelman
Principal
Investigator



Philip
Julia Expert



Chris
Rackauckas
Expert
Collaborator



Bob Metcalfe
Research
Affiliate



Virginia
Spanoudaki
Senior
Research



Emmanuel
Lujan
Research
Scientist



Rabab
Alomairy
Postdoctoral
Fellow



Avik Pal
PhD Student



Bowen Zhu
PhD Student



Emily Helen
Nieves
PhD Student



Nicholas E
Klugman
PhD Student



Utkarsh
PhD Student



Evelyne
Ringoot
PhD Student



Raye Kimmerer
Master's
Student



Songchen Tan
PhD Student



Vaibhav Dixit
Master's
Student



Julian Samaroo
Research
Software
Engineer



Blanca Luo
Undergraduate



Collin
Wittenstein
Visiting
Master's
Student



Theodoros
Xenakis
Visiting
Master's
Student



Artemis Pados
Undergraduate

Less Code, More Performance: Abstracting Singular Values Across Precision, & Hardware

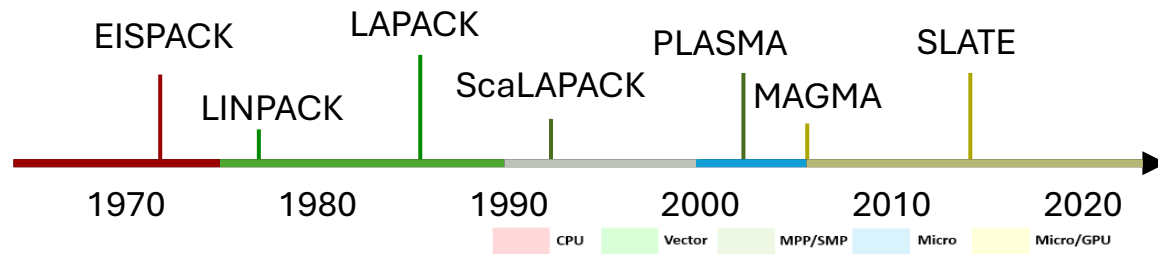
COMPSCI2050 – April 2, 2026

Speaker: **Evelyne Ringoot**

Other contributors: Rabab Alomairy, Alan Edelman, Valentin Churavy, James Schloss, Julian Samaroo, Sophie Xuan, Vicki Carrera, Max Onyango



State of the art HPC linear algebra library landscape is convoluted: redevelopment for every hardware



(adapted from Reed, Dannon, and Dongarra, 2022)



"[With every new hardware] we scramble for the next three or four years to figure out how to use it effectively, [...] an incredibly intensive process of redesigning algorithms"

-Dongarra, 2024

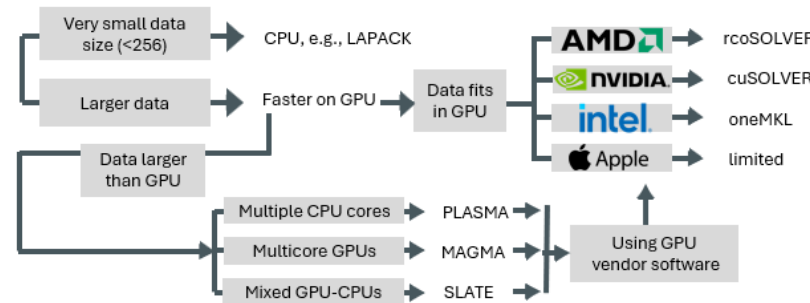
➔ Application libraries typically cannot support all hardware set-ups (e.g. PyTorch)

➔ Large HPC clusters with **heterogeneous hardware** require implementations of several libraries

(Dufek 2021, Godoy 2023, Malenza 2024)

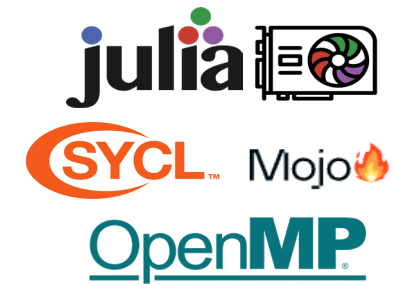
Less code, more performance: abstracting SVD (and linear algebra)

Today: separate low-level functions by hardware and data precision



Heterogeneous hardware? Mixed precision?

Abstraction frameworks



Less code, more performance: abstracting SVD (and linear algebra)

Seamless support for



Novel hardware
(e.g. Apple Metal)

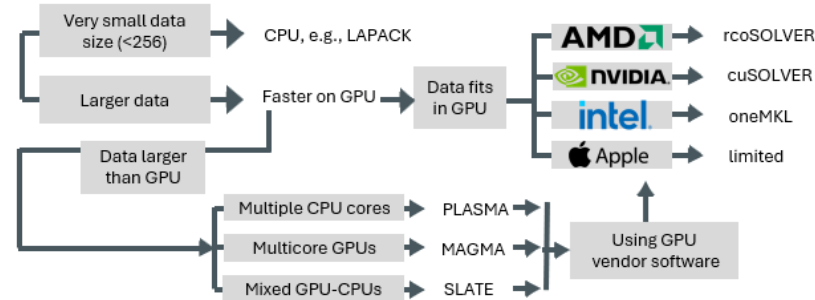


Novel data precision
or mixed precision
(e.g. FP16)



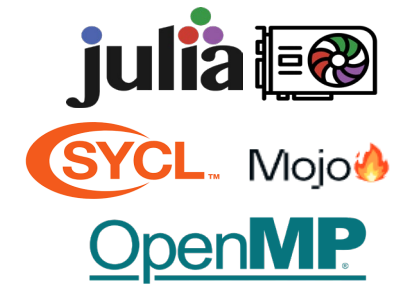
HPC code development
on laptops

Today: separate low-level functions by
hardware and data precision



Heterogeneous hardware? Mixed precision?

Abstraction frameworks



Less code, more performance: abstracting SVD (and linear algebra)

Seamless support for

Today: separate low-level functions by
hardware and data precision



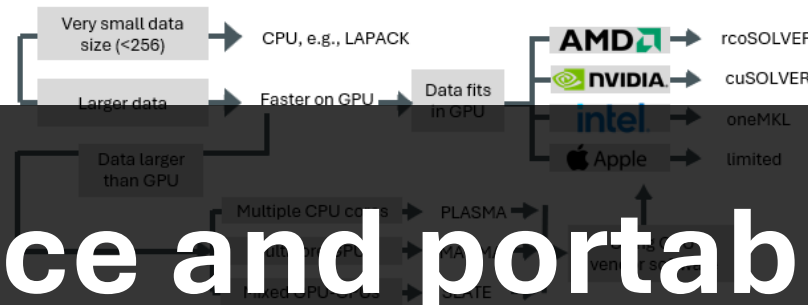
Novel hardware
(e.g. Apple Metal)



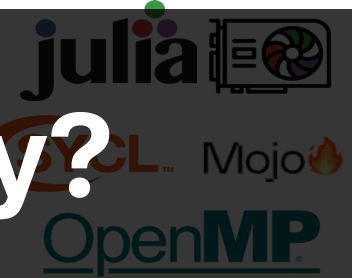
Not all a process or mixed precision
(e.g. FP16)



HPC code development
on laptops



Abstraction frameworks



Performance and portability?

Heterogeneous hardware? Mixed precision?

Less code, more performance: abstracting SVD (and linear algebra)

Seamless support for



Novel hardware
(e.g. Apple Metal)

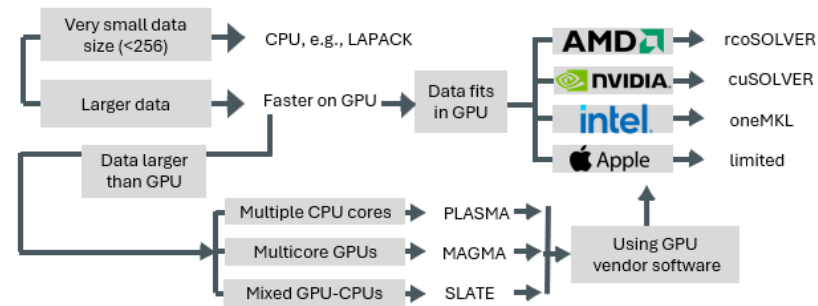


Novel data precision
or mixed precision
(e.g. FP16)



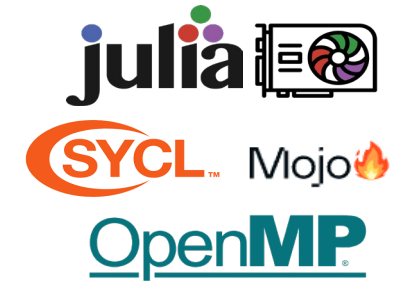
HPC code development
on laptops

Today: separate low-level functions by hardware and data precision

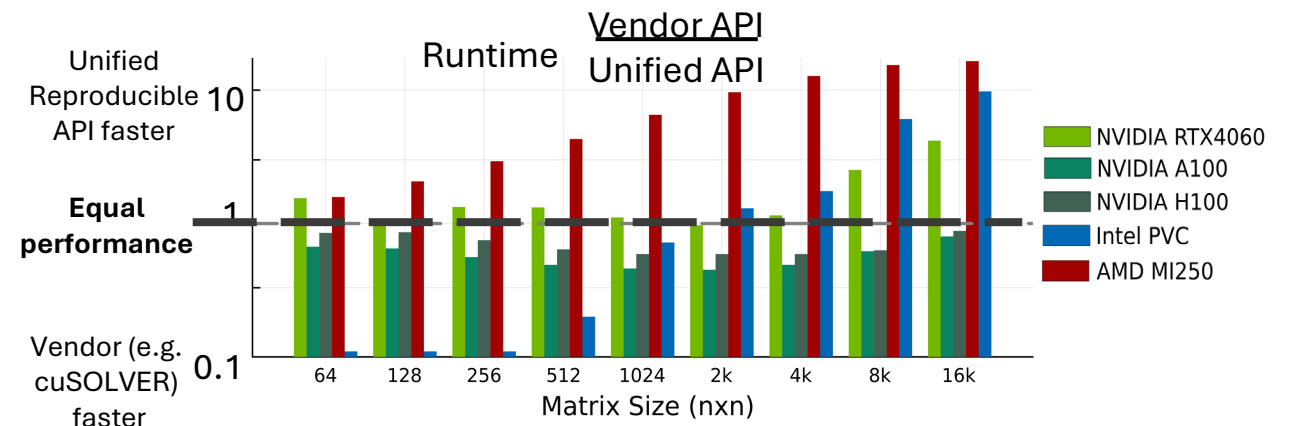


Heterogeneous hardware? Mixed precision?

Abstraction frameworks

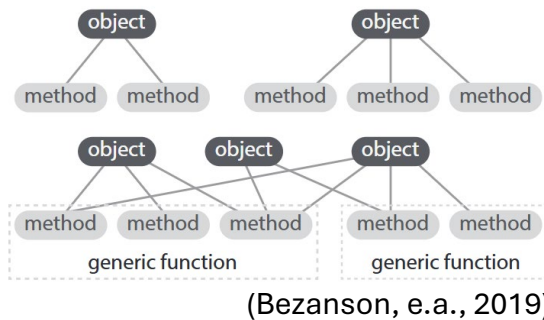


Performance and portability?



Julia language features provide composability across hardware and data types

Multiple-dispatch provides composability



High-level abstractions allow plugging in low-level building blocks and extending to new building blocks seamlessly:

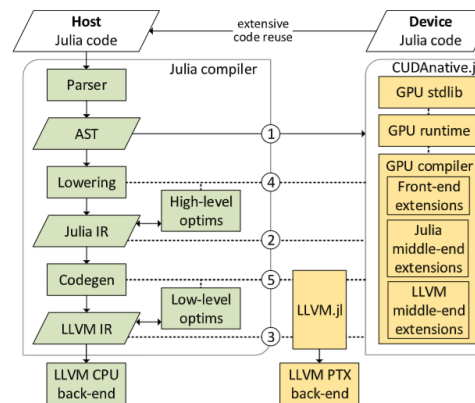


(library: Besard, e.a., 2025, Samaroo, e.a., 2025)

Array abstractions: portable across CPU and GPU

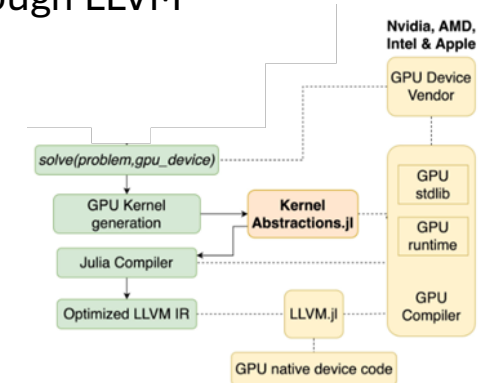
Hardware-interactions with CPU and GPU

(Besard, e.a., 2019)



Kernel abstractions: portable code across GPU vendors

Kernels compiled to native code through LLVM

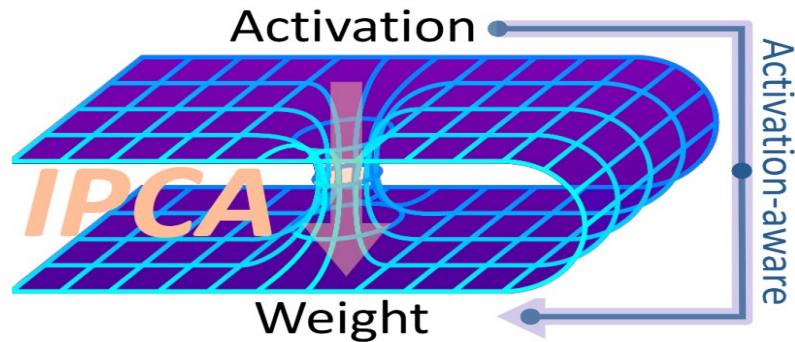


(Utkarsh, e.a., 2023,
library: Churavy, e.a., 2023)

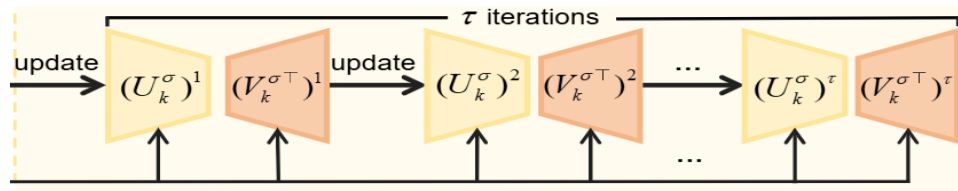
➔ Building on GEMM for BLAS linear algebra (Boisvert e.a. 1987, BLIS Van Zee, e.a.)

Singular value decomposition is cornerstone of linear algebra, including LoRa for LLMs

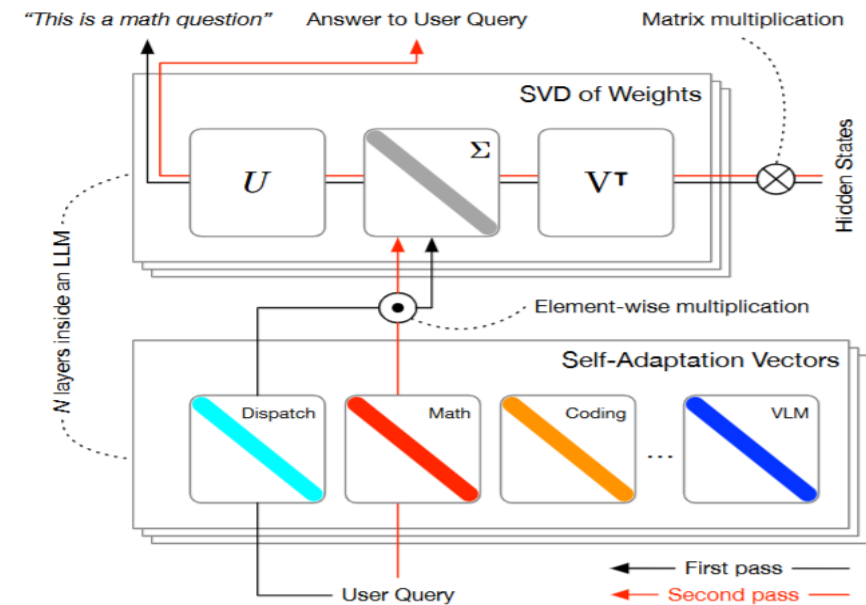
E.g. DOBI-SVD (Wang, e.a., 2025)



E.g. ADA-SVD (Li, e.a., 2024)



Transformer squared
(Sun, e.a., 2024)

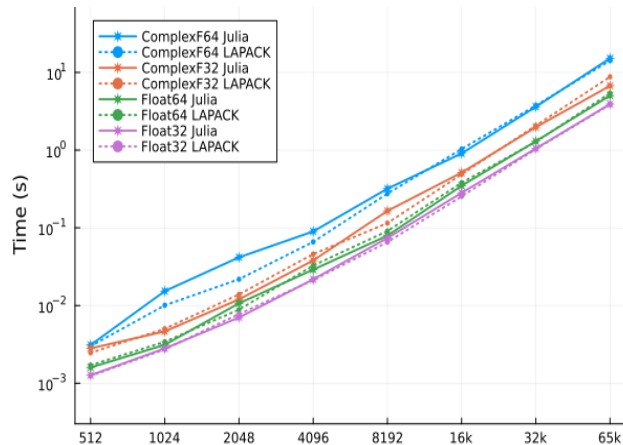


One implementation of linear algebra across all dimensions without sacrificing performance

1

Across hardware & precision

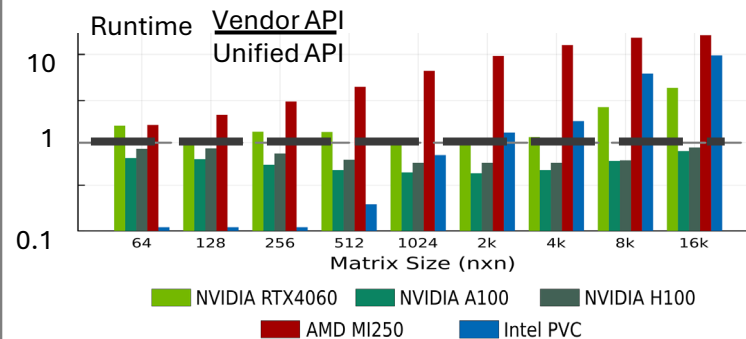
Relying on low-level vendor functions, enabling mixed-precision



2

Across GPU vendors

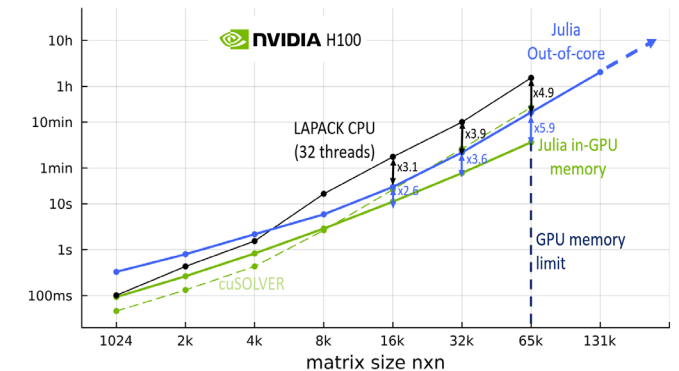
Replacing low-level vendor functions



3

Across multi-hardware

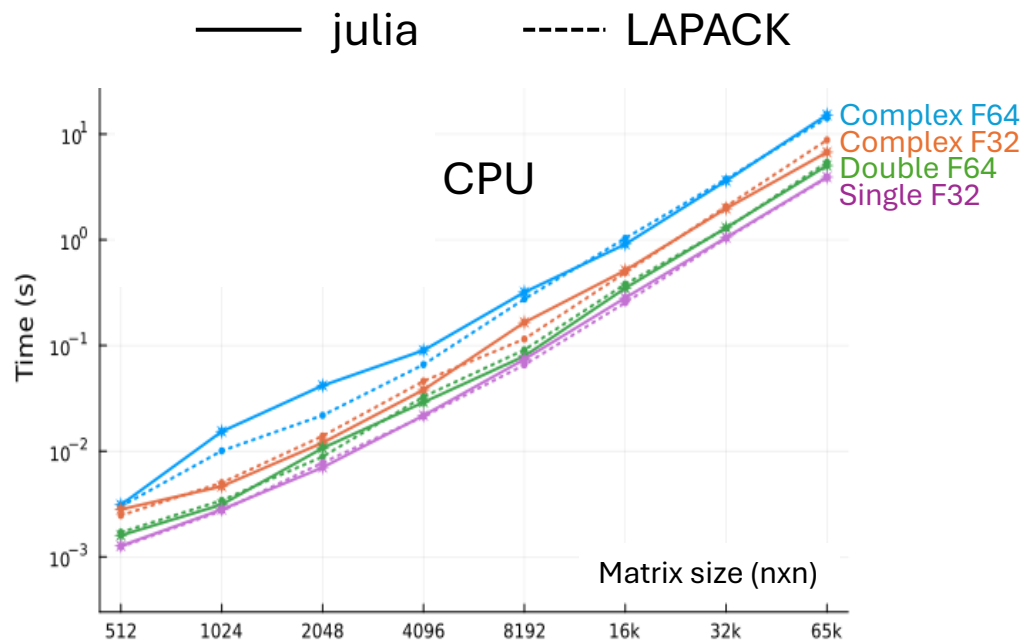
Replacing high-level large-scale HPC libraries



1

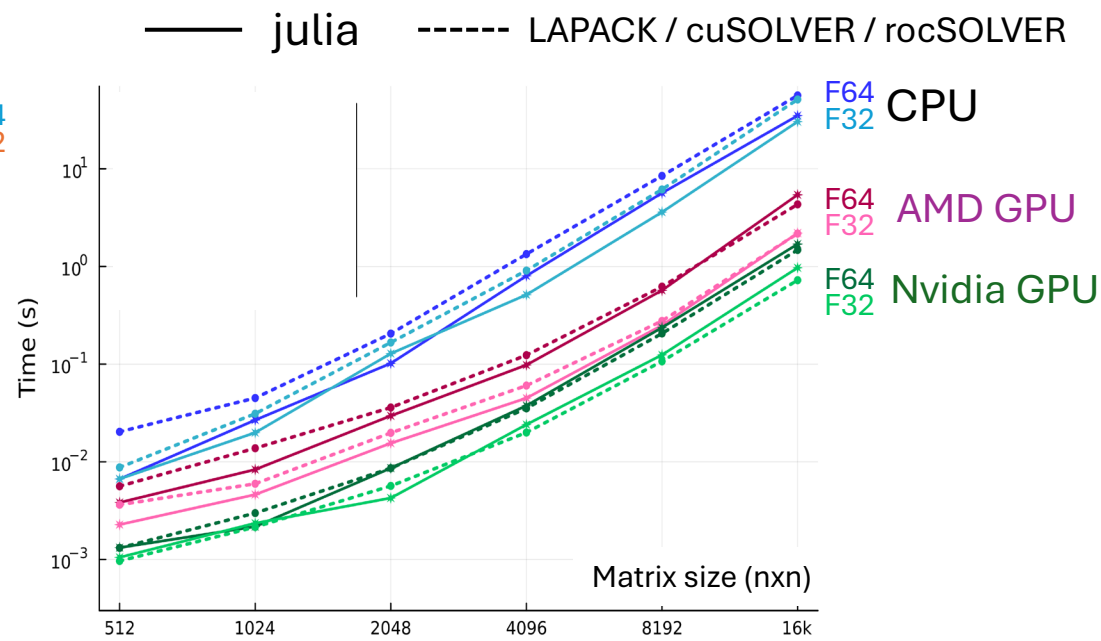
Abstraction for QR/LU matches LAPACK&cuSOLVER performance across data types and hardware

Across Data types



larfb: $A + AYT Y^T = AQ$

Across CPU-GPU Hardware

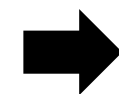
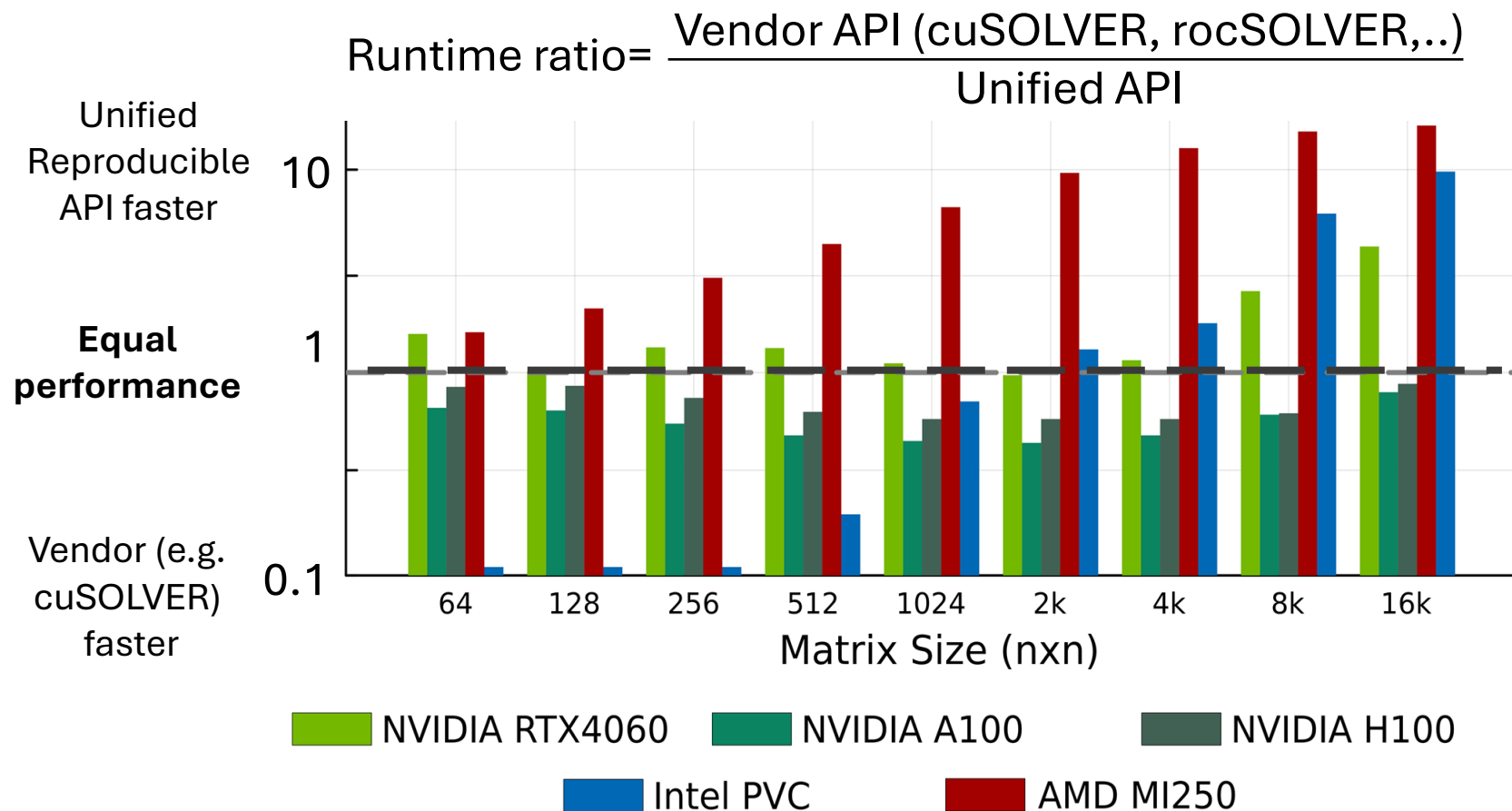


urmqr: $A + YTY^T A = QA$

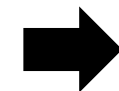
2

Abstract GPU kernels perform similar or better than vendor libraries, enabling new data types and hardware types

Singular values solver



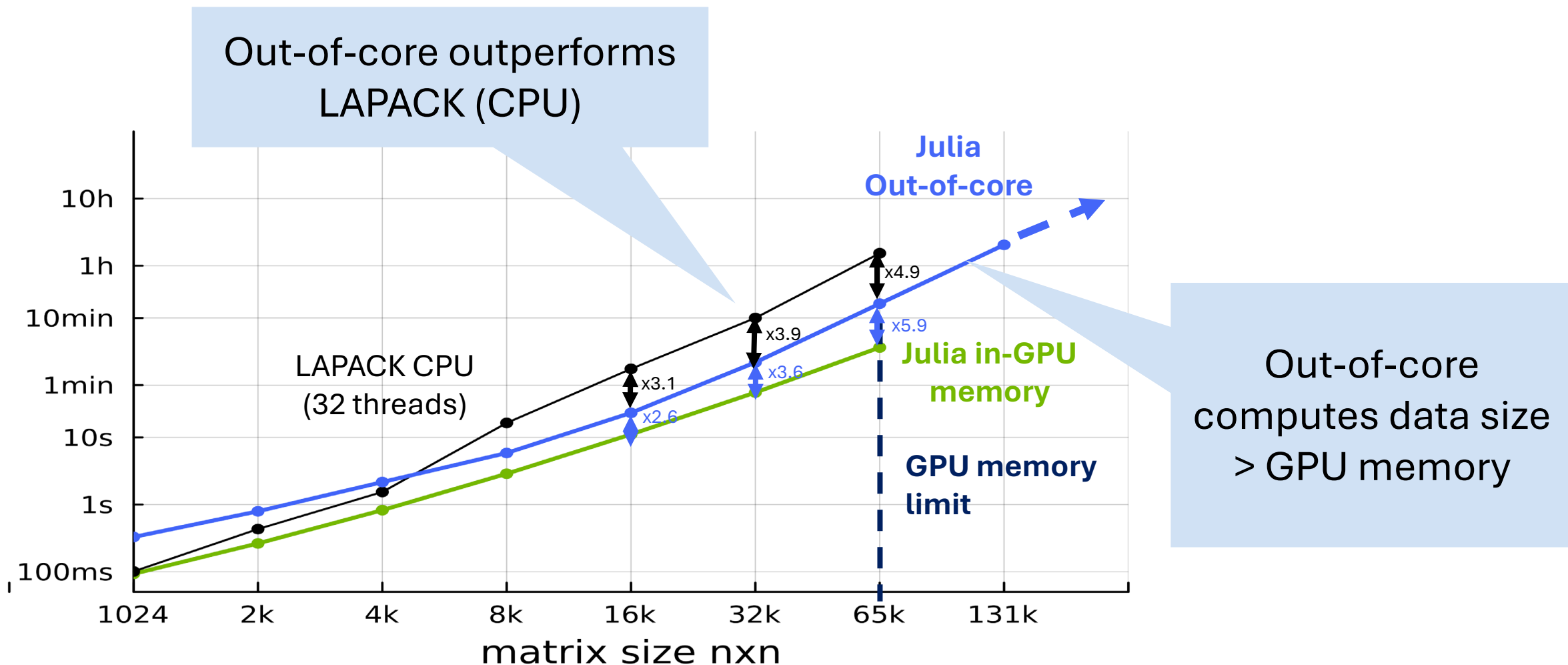
First half
precision FP16
singular values



First Apple Metal
singular values

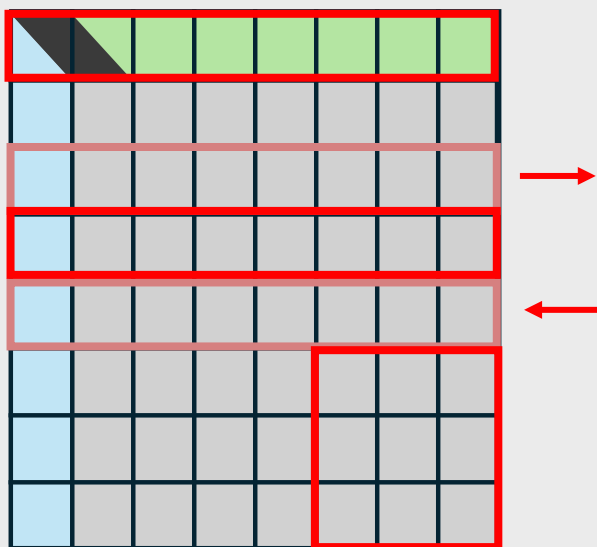
3

Abstraction across in-GPU and CPU-GPU: out-of-core singular values performance in line with GPU performance



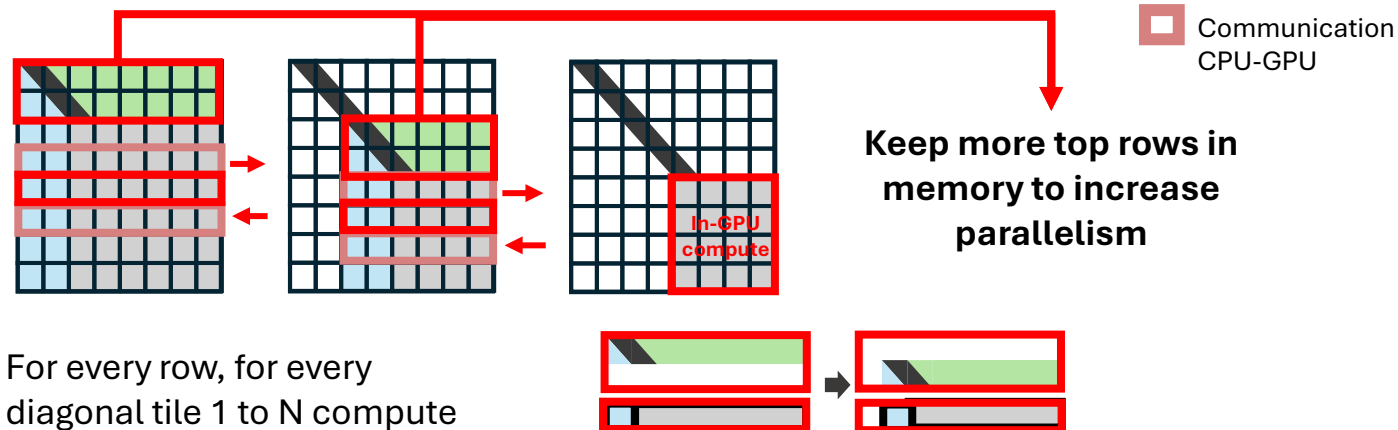
3

Optimizing communication by hardware



Prioritize communication-computation overlap over communication-minimization

For **every N** diagonal tile, for **every row** communicate



Single high-level function governing algorithm with multiple-dispatch seamlessly integrating communication

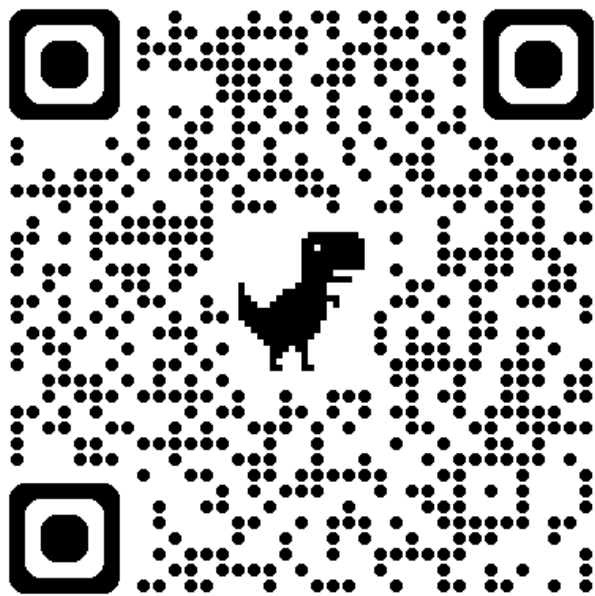
```
function banddiag!(A::AbstractLargeGPUMatrix,
  Tau::AbstractGPUMatrix, nbtiles::Int, backend)
  for k in 1:(nbtiles-1)
    GETSMQRT!(A,Tau,k, nbtiles, backend)
    GETSMQRT!(A',Tau,k, nbtiles, backend, LQ=true)
  end
  GETSMQRT!(A,Tau,nbtiles, nbtiles, backend)
  return A
end

function GETSMQRT!(A::AbstractGPUMatrix, ...)
  GEQRT!(A,k)
  UNMQR!(A, k)
end

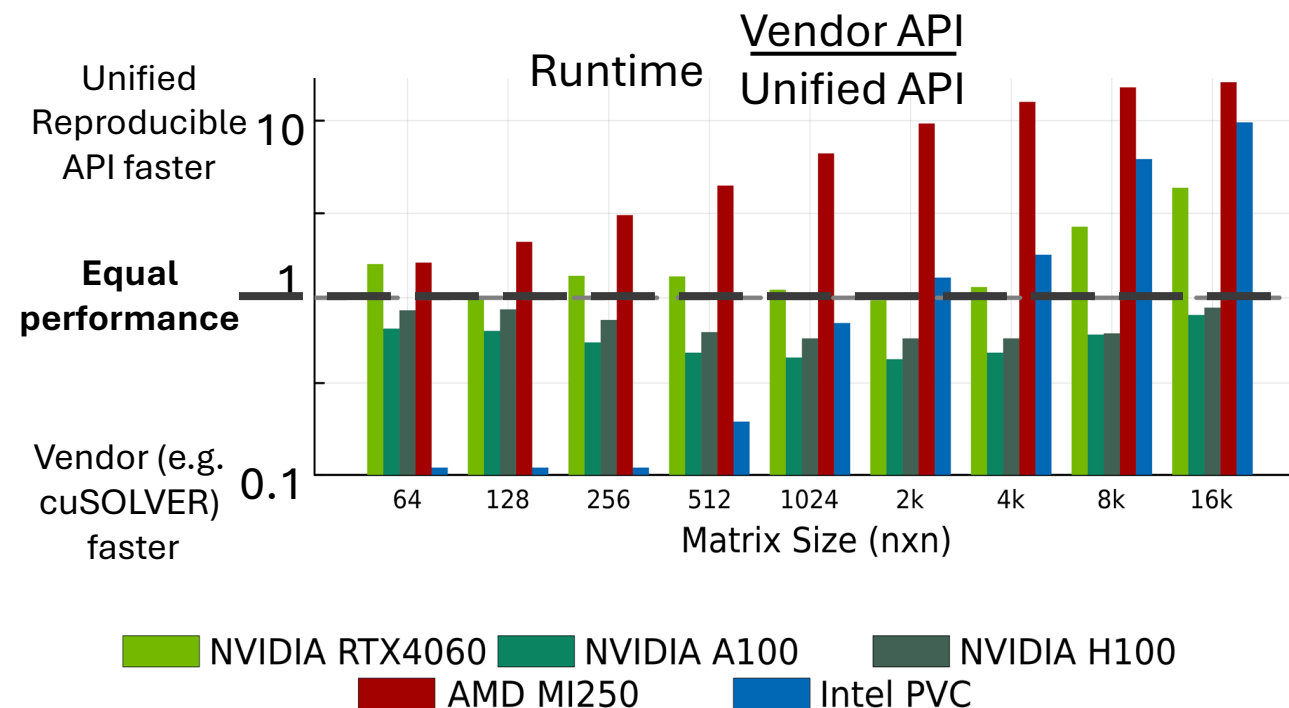
function GETSMQRT!(A::LargeTiledMatrix, ...)
  ... #communication CPU-GPU
  GEQRT!(A.data[k],...)
  UNMQR!(A.data[k],...)
  KernelAbstractions.@sync
end
```

async

Open-source abstract linear algebra library NextLA.jl



Performance and portability!



EXAMPLE 1

EXAMPLE 2

EXAMPLE 3